



# RTM View Shell

## Multi-Tenant Management Guide

*Tenant lifecycle, settings, agent-state model, SSO, RTM binding & isolation*

Administrator guide A-07 — for the Superadmin (platform operator)

Edition EN · 2026-07-03 · RTM View Shell

## Document revision history

This table lists the revision history of this document.

| Version | Date       | Summary of changes                                                          | Product release ID | Shipped-with (commit/barrier) |
|---------|------------|-----------------------------------------------------------------------------|--------------------|-------------------------------|
| 1.0     | 2026-06-13 | Initial                                                                     | RTM-REL-2026.06    | (pending push)                |
| 1.1     | 2026-06-22 | Added<br>SIThresholdSeconds<br>tenant setting<br>(Historical Reports<br>SL) | RTM-REL-2026.06    | (pending push)                |

## Contents

|                                                                |   |
|----------------------------------------------------------------|---|
| Document revision history.....                                 | 2 |
| Contents.....                                                  | 3 |
| 1. Overview — what “multi-tenant” means here.....              | 4 |
| 2. The tenant lifecycle.....                                   | 4 |
| 3. Tenant Settings.....                                        | 5 |
| 4. Appearance & localisation.....                              | 6 |
| 5. The agent-state model.....                                  | 6 |
| 6. SSO configuration (per tenant).....                         | 6 |
| 7. Binding an RTM Service instance to a tenant.....            | 7 |
| 8. Tenant switching (impersonation).....                       | 7 |
| 9. Isolation & security guarantees (the compliance story)..... | 7 |
| 10. Auditing tenant operations.....                            | 8 |
| Appendix A — Tenant Settings reference.....                    | 9 |
| Appendix B — Agent-state seed & validation.....                | 9 |
| Appendix C — RTM Service per-tenant deployment checklist.....  | 9 |
| Appendix D — Audit event catalogue (tenant operations).....    | 9 |

## 1. Overview — what “multi-tenant” means here

RTM View Shell hosts many customer organisations (“tenants”) on one installation. The model is shared database, shared schema: every tenant-scoped row carries a TenantId discriminator, and the application automatically filters every query by the current tenant. A tenant is the unit of isolation — its users, permission groups, dashboards, queues, skills and agent-state model are private to it.

Three facts frame everything in this guide:

- **One Superadmin governs the whole platform.** The Superadmin is the only role that sees across tenants, creates them, and can switch into any one of them (impersonation, always audited). Every other role — Administrator, Editor, Viewer — is confined to a single tenant.
- **Isolation is enforced in code, not just the UI.** Global Query Filters scope every database read to the active tenant; hiding a menu is cosmetic only. §9 documents this in full.
- **One RTM Service instance serves exactly one tenant.** The data-collection Windows Service is bound to a tenant at deployment time through its configuration file. Binding is covered in §7.

Tenant management lives on Screen 06 — Tenant Management (/platform/tenants), visible to the Superadmin only. Its four tabs — General, Settings, Appearance, Agent States — map to §2–§5 below.

## 2. The tenant lifecycle

A tenant is always in exactly one of three states (Tenant.Status):

| State     | Meaning              | Effect on login                    | Data                              |
|-----------|----------------------|------------------------------------|-----------------------------------|
| Active    | Normal operation     | Logins permitted                   | Live                              |
| Suspended | Temporarily disabled | All logins for the tenant rejected | Retained intact                   |
| Deleted   | Decommissioned       | Invisible to authentication        | Purged ≥ 30 days after transition |

Each transition writes an audit event (Tenant.Created, Tenant.Suspended, Tenant.Resumed, Tenant.Deleted), so the lifecycle is fully traceable (§10).

**Create a tenant** (General tab → “+ New Tenant”): supply a display Name and a unique Slug. The slug becomes the tenant’s subdomain — acme resolves acme.cc-dashboard.local to this tenant — so it must be lowercase, unique, and stable. On first save the platform seeds the tenant: its tenant\_settings row with defaults (§3), the five standard agent-state definitions (§5), and an empty permission-group set.

**Suspend / Resume** (General tab → Status dropdown): set Suspended to immediately block every login without touching data; set it back to Active to restore. Existing sessions are rejected on their next authenticated action.

**Delete** (General tab → Status → Deleted): a soft transition. The tenant disappears from authentication immediately, but its data remains on disk for at least 30 days before the background purge job removes it.

**Worked walkthrough — stand up tenant “acme”**

1. General tab → “+ New Tenant” → Name “Acme Corp”, Slug acme, Status Active → Save. The platform writes the tenants row, a default tenant\_settings row, seeds five agent-state definitions; Tenant.Created lands in the audit log.
2. Settings tab → set the password policy and 2FA requirement for Acme (§3).
3. Create Acme’s first Administrator (User Management, A-02) — that Administrator then builds Acme’s permission groups, users and dashboards.
4. When Acme’s RTM Service is deployed, copy Acme’s tenant UUID from this screen into the service config (§7). Acme is now live and fully isolated.

### 3. Tenant Settings

The Settings tab edits the tenant’s tenant\_settings row (one-to-one with the tenant). Every value here applies only to this tenant:

| Setting                                | Field                   | Default   | Effect                                                                                 |
|----------------------------------------|-------------------------|-----------|----------------------------------------------------------------------------------------|
| Licensing — purchased licences         | integer                 | —         | Informational seat cap                                                                 |
| Licensing — user connections           | integer                 | 0         | Max concurrent connections; 0 = unlimited                                              |
| Password — minimum length              | PasswordMinLength       | 12        | Enforced on every password set                                                         |
| Password — expiry (days)               | PasswordExpireDays      | 90        | Forces a change after N days                                                           |
| Security — require 2FA for all         | Require2faForAll        | off       | Forces e-mail 2FA on for every user                                                    |
| Security — default locale              | DefaultLocale           | en-US     | BCP-47 culture for new users / login                                                   |
| SignalR Widgets — connection URL       | SignalRConnectionUrl    | —         | RTM Service hub URL the relay connects to (§7)                                         |
| Data — audit retention (days)          | AuditRetentionDays      | 365       | Age beyond which audit partitions drop                                                 |
| Data — soft-delete screens             | SoftDeleteDashboards    | off       | Deleting a dashboard hides vs removes                                                  |
| Data — soft-delete retention (days)    | SoftDeleteRetentionDays | 90        | Age after which soft-deleted dashboards purge                                          |
| Service level — SL threshold (seconds) | SLThresholdSeconds      | — (unset) | Per-tenant service-level target in seconds; used by Historical Reports SL calculations |

The password and 2FA settings feed the security behaviour documented in the Installation & Security guides; the SignalR connection URL is the single binding between this tenant and its RTM Service hub (§7). Sensitive provider fields (e-mail credentials, SSO client secret) are stored encrypted.

## 4. Appearance & localisation

The Appearance tab sets per-tenant presentation defaults that the (future) widget editor inherits: available font sizes, a background colour palette, and a font colour palette. These do not affect isolation or security — they brand the tenant's screens.

Localisation is driven by Default locale (Settings tab) plus each user's own PreferredLocale. The platform supports any BCP-47 language by adding a resource file — no code change — and renders right-to-left layouts automatically for Arabic, Hebrew and Farsi. All timestamps are stored in UTC and converted to the user's locale in the UI.

## 5. The agent-state model

The Agent States tab (Superadmin-only) defines how the raw agent-status names from the contact-centre platform are grouped for display and metric aggregation. Three concepts:

- **Agent State** — a raw state name as the CC platform emits it (e.g. AVAILABLE, LUNCH, ONPHONE).
- **State Group** — a display bucket (e.g. Available, Break) that one or more states roll up into.
- **Definition** — the mapping that assigns exactly one State to exactly one Group, per tenant.

The metric identifier is not stored on the definition; it is resolved at query time by joining the metric catalogue on the state name. Every tenant is seeded with five standard definitions: AVAILABLE → Available, ONPHONE → On Phone, BREAK → Break, PAPERWORK → Paperwork, TRAINING → Training.

**Managing groups** (Section 1): add a group, rename it inline, or deactivate it (Danger-Zone: reassign the group's states to another active group, or deactivate all its states — you must choose).

**Managing states** (Section 2): add a state (name + a target active group), change a state's mapped group, or deactivate a state (this also deactivates its definition). Nothing is ever physically deleted; deactivation is a soft state.

**Validation:** state names and group names are each unique per tenant, case-insensitive; a state cannot exist without being assigned to a group.

### Worked walkthrough — add a “LUNCH” state

Acme's platform emits a distinct LUNCH status that should report under breaks but separately from short breaks.

1. Section 1 → “+ Add State Group” → Meal Break.

2. Section 2 → “+ Add State” → name LUNCH, group Meal Break → Save.

From the next refresh, time in LUNCH aggregates under Meal Break in every agent-state metric for Acme — no code change, no effect on other tenants.

## 6. SSO configuration (per tenant)

Each tenant may have its own single-sign-on provider, stored in `sso_configurations` and linked from the tenant's settings. Supported protocols: SAML 2.0, OIDC, and AD / LDAP. You provide the provider metadata and a claim-mapping that translates the provider's claims into local user fields; on SSO login a local user is provisioned just-in-time. If the provider asserts MFA (`amr=mfa`), system 2FA is skipped.

### Version note

In the current release SSO is a configurable stub: the screens accept and store a configuration, but live federation is delivered in a later sprint. Until then, users sign in with local accounts plus e-mail two-factor. The configuration you enter now is preserved and activates when the provider integration ships.

## 7. Binding an RTM Service instance to a tenant

The RTM Service collects real-time data and pushes it to the Shell. It has no logged-in user, so it is bound to a tenant statically, at deployment. One instance = one tenant. Two halves:

1. **In the service config** (appsettings.json, section RTM): set "TenantId" to the tenant's UUID. Empty/missing = fatal start-up error.
2. **In the Shell** (Settings tab → SignalR Connection URL): set the tenant's hub URL; the Shell's relay opens one server-to-server connection (browsers never connect to the service directly).

Deployment checklist: (1) copy the tenant UUID from Screen 06; (2) set RTM:TenantId before first start; (3) confirm the bound TenantId in the start-up log; (4) verify the tenant's business units are visible and non-zero where data exists.

### Safety note — never run an unscoped midnight clear

The service's daily housekeeping deletes interaction and user-status rows. It must be tenant-scoped (WHERE "TenantId" = ...); an unscoped version would wipe all tenants. Confirm the deployed routine is the tenant-scoped build before enabling the schedule.

*For the full install / upgrade mechanics of the service, see A-01 — Installation & Deployment Guide.*

## 8. Tenant switching (impersonation)

The Superadmin can switch the active tenant from the admin UI to operate inside a tenant's context — to reproduce a problem, verify a configuration, or assist support. Switching changes the TenantId on the active session and writes a Tenant.Switched audit event, so every impersonation is on the record. Switch back the same way when finished.

## 9. Isolation & security guarantees (the compliance story)

- **Automatic query scoping.** Every read on a tenant-scoped table is filtered by the active TenantId through Global Query Filters. Application code cannot accidentally read another tenant's rows.
- **Audited escape hatch only.** The filter is bypassed solely inside dedicated system repositories, and every such use writes a Tenant.CrossTenantAccess audit event.
- **Defence in depth.** Cache keys, real-time group names and blob containers are all tenant-prefixed — isolation holds in Redis, SignalR and storage, not only the database.
- **Intentionally shared platform data.** The widget catalogue, application roles and the tenant registry are cross-tenant and carry no customer data.
- **Login binds user to tenant.** Authentication verifies the user's tenant matches the resolved subdomain; a mismatch is rejected with the generic failed-login message and recorded as Login.Failure / TenantMismatch.

## 10. Auditing tenant operations

Every platform-level action is recorded and viewable on the Audit screen. Tenant-relevant events: Tenant.Created/Suspended/Resumed/Deleted; Tenant.Switched; Tenant.CrossTenantAccess; WidgetCatalog.ItemAdded/ItemUpdated/ItemDeactivated; System.AuditPurged. Each entry carries actor, UTC timestamp, IP, user-agent, JSON details; retained per AuditRetentionDays (default 365). Exports over 50,000 rows run as a background job delivered by e-mail.



## Appendix A — Tenant Settings reference

The full tenant\_settings column set, types, defaults and effect — see the table in §3 (incl. SThresholdSeconds). Encrypted fields (e-mail provider config, SSO client secret) are protected at rest and never shown in plaintext.

## Appendix B — Agent-state seed & validation

Seed (every tenant, first run): AVAILABLE → Available, ONPHONE → On Phone, BREAK → Break, PAPERWORK → Paperwork, TRAINING → Training. Validation: state/group names unique per tenant (case-insensitive); one state → one group; no state without a group; deactivation is soft.

## Appendix C — RTM Service per-tenant deployment checklist

Copy the tenant UUID (Screen 06) → set RTM:TenantId (empty = fatal) → set the SignalR Connection URL in the Shell → start; confirm bound TenantId in the log → verify data is scoped (non-zero BU count) → confirm the midnight-clear routine is tenant-scoped before enabling the schedule.

## Appendix D — Audit event catalogue (tenant operations)

Tenant.Created, Tenant.Suspended, Tenant.Resumed, Tenant.Deleted, Tenant.Switched, Tenant.CrossTenantAccess, WidgetCatalog.ItemAdded, WidgetCatalog.ItemUpdated, WidgetCatalog.ItemDeactivated, System.AuditPurged. Each: actor, UTC timestamp, IP, user-agent, JSON details; retained per AuditRetentionDays.